Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Secuencias y Transformer

Módulo 12

Representar el lenguaje

Tokenización, embeddings y modelos secuenciales

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 12: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
Σ, Σ^*	alfabeto de caracteres (puntos de código Unicode) y cadenas sobre él
$\mathcal{B} = \{0, \dots, 255\}$	conjunto de los bytes; $\mathcal{B}^*$, cadenas de bytes
`U+00F1`, `C3`, `110xxxxx`	punto de código, byte en hexadecimal y patrón de bits
$\mathcal{V}$, $K = \lvert\mathcal{V}\rvert$, $\mathcal{V}_0$	vocabulario, su tamaño y vocabulario inicial de BPE
k	número de fusiones de BPE
$u \Vert v$	concatenación de las cadenas u y v
$s = (t_1, \dots, t_m)$, $p(t)$	segmentación en tokens y probabilidad unigram de un token
$E \in \mathbb{R}^{K \times d}$, e_i	matriz de embeddings y vector *one-hot* del token i
v_w, u_w	vector de una palabra como centro y como contexto en skip-gram
σ, $\odot$	sigmoide logística y producto componente a componente
h_t, c_t	estado oculto de una red recurrente y celda de memoria de una LSTM
f_t, i_t, o_t	puertas de olvido, de entrada y de salida de una LSTM
$\lVert\cdot\rVert$	norma euclídea de un vector o norma espectral de una matriz

Índice

Sobre este resumen **ii**

1 Del texto a los bytes: Unicode y UTF-8 **1**
1.1 El problema de la tokenización . . . 1
1.2 Unicode y puntos de código . . . 1
1.3 La codificación UTF-8 . . . 2
1.4 Propiedades de UTF-8 . . . 2
1.5 Consecuencias para el español y para los LLM . . . 3

2 El algoritmo BPE **4**
2.1 Entrenamiento . . . 4
2.2 Codificación y decodificación . . . 5
2.3 Desempates y determinismo . . . 5
2.4 BPE a nivel de byte y pretokenización . . . 6

3 El modelo unigram, SentencePiece y el vocabulario **7**
3.1 El modelo unigram . . . 7
3.2 Segmentación óptima: el algoritmo de Viterbi . . . 7
3.3 Entrenamiento por EM y poda . . . 8
3.4 SentencePiece . . . 8
3.5 El tamaño del vocabulario . . . 9
3.6 Efectos de la tokenización . . . 10

4 Embeddings y word2vec **11**
4.1 La matriz de embeddings . . . 11
4.2 La hipótesis distribucional . . . 11
4.3 Skip-gram . . . 12
4.4 Muestreo negativo . . . 13
4.5 Muestreo negativo y PMI . . . 13
4.6 Embeddings en un LLM . . . 13

5 Modelos secuenciales: n-gramas, RNN y LSTM **14**

5.1 N-gramas y dispersión 14

5.2 Redes recurrentes 14

5.3 Retropropagación en el tiempo y desvanecimiento del gradiente 15

5.4 La LSTM 16

5.5 Secuencialidad y el paso al Transformer 16

Bibliografía **17**

Tema 1

Del texto a los bytes: Unicode y UTF-8

Un tokenizador transforma texto en una secuencia de enteros. Antes de elegir las unidades conviene fijar qué es un texto para un ordenador: una sucesión de puntos de código Unicode que se almacena como una sucesión de bytes. Este tema plantea el compromiso entre longitud de la secuencia y tamaño del vocabulario, define la codificación UTF-8 y demuestra las dos propiedades que la hacen adecuada como punto de partida de los tokenizadores modernos: es un código prefijo y se autosincroniza. Se sigue [10, §2.3], la definición del RFC 3629 [22] y la §2 de la tarea 1 de CS336 [4].

1.1 El problema de la tokenización

Se recuerda del módulo 1 que un *tokenizador* es un par de aplicaciones $\tau\colon \Sigma^* \to \mathcal{V}^*$ y $\delta\colon \mathcal{V}^* \to \Sigma^*$ con $\delta(\tau(s)) = s$ para toda cadena s. La elección de $\mathcal{V}$ fija la granularidad y, con ella, dos magnitudes que compiten: el tamaño $K = |\mathcal{V}|$ y la longitud $|\tau(s)|$ de las secuencias.

Cuadro 1.1: Granularidad de la tokenización, con la palabra «caballerías» como ejemplo.

Unidad	Tokens	K típico	Limitación principal
Carácter	11	$\sim 10^2$	secuencias largas; la atención cuesta $O(n^2)$
Palabra	1	$> 5 \cdot 10^5$	palabras fuera del vocabulario; formas sin relación
Subpalabra	2–5	$3{,}2 \cdot 10^4$–$2{,}6 \cdot 10^5$	hay que entrenar el tokenizador

Con un vocabulario de palabras, «caballería» y «caballerías» son tokens sin ninguna relación, y una palabra nueva o con una errata no tiene representación. Con caracteres, cualquier texto es representable, pero las secuencias son largas, y el coste de la atención crece con el cuadrado de la longitud (módulo 13). Las subpalabras reparten el vocabulario según la frecuencia: las palabras comunes son un solo token y las raras se componen de piezas frecuentes.

Observación 1.1. El tokenizador se entrena antes que el modelo y por separado, con un algoritmo de conteo, y después se congela. El modelo solo recibe los enteros que produce. En los temas 2 y 3 se estudian los dos algoritmos de entrenamiento más usados.

1.2 Unicode y puntos de código

Definición 1.2 (Punto de código). Unicode asigna a cada carácter un entero, su *punto de código*, del conjunto $\{0, 1, \ldots, \texttt{10FFFF}_{16}\}$, que se escribe en hexadecimal con el prefijo `U+`. Los puntos de código del intervalo `U+D800`–`U+DFFF` (*sustitutos*) están reservados y no representan caracteres. Los restantes se llaman *valores escalares Unicode* [22].

Por ejemplo, «a» es `U+0061` (97), «ñ» es `U+00F1` (241) y el signo del euro es `U+20AC` (8364). Un punto de código es abstracto: no fija la forma gráfica (el glifo) con que se dibuja el carácter [10, §2.3.1]. Unicode 16.0 define más de $1{,}5 \cdot 10^5$ caracteres.

Un modelo podría usar los puntos de código como vocabulario inicial, pero este tendría más de $1{,}3 \cdot 10^5$ símbolos, casi todos raros. Por eso los tokenizadores modernos parten de la representación del texto en bytes.

1.3 La codificación UTF-8

Definición 1.3 (UTF-8). La codificación *UTF-8* asigna a cada valor escalar c una cadena de 1 a 4 bytes según el cuadro 1.2. Los bits de c, escritos en binario con el número de bits útiles de su fila y completados con ceros a la izquierda, rellenan de izquierda a derecha las posiciones marcadas con `x`. La codificación de un texto es la concatenación de las codificaciones de sus caracteres.

Cuadro 1.2: Definición de UTF-8 [22]. Se llama *byte inicial* al primero de cada fila y *bytes de continuación* a los demás.

Puntos de código	Bytes	Bits útiles	Patrón de bits
`U+0000`–`U+007F`	1	7	`0xxxxxxx`
`U+0080`–`U+07FF`	2	11	`110xxxxx 10xxxxxx`
`U+0800`–`U+FFFF`	3	16	`1110xxxx 10xxxxxx 10xxxxxx`
`U+10000`–`U+10FFFF`	4	21	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

Se exige además que cada carácter use la fila más corta en la que cabe: las codificaciones *sobrelargas*, como escribir «a» con dos bytes, no son UTF-8 válido. Como consecuencia, los bytes `C0`, `C1` y `F5`–`FF` no aparecen nunca en un texto UTF-8 [22].

Proposición 1.5. *El número de puntos de código que UTF-8 codifica con 1, 2, 3 y 4 bytes es, respectivamente,* 128*,* 1920*,* 63 488 *y* 1 048 576*. Descontando los* 2048 *sustitutos, que pertenecen a la fila de 3 bytes, hay* 1 112 064 *valores escalares.*

1.4 Propiedades de UTF-8

Definición 1.6 (Código prefijo). Sea $C\colon \Sigma \to \mathcal{B}^*$ una aplicación que asigna a cada símbolo una cadena de bytes no vacía. C es un *código prefijo* si ninguna palabra del código $C(a)$ es prefijo propio de otra $C(b)$ y, además, C es inyectiva.

Teorema 1.7 (UTF-8 es un código prefijo). *La codificación UTF-8 de un carácter es inyectiva y ninguna palabra del código es prefijo propio de otra. En consecuencia, su extensión a cadenas,* $\mathrm{enc}\colon \Sigma^* \to \mathcal{B}^*$*, es inyectiva: toda cadena de bytes UTF-8 válida se descompone de forma única en caracteres.*

Proposición 1.8 (Autosincronización). *En una cadena UTF-8 válida, un byte es de continuación si y solo si sus dos bits más altos son* `10`*. Por tanto, desde cualquier posición se encuentra el comienzo del carácter que la contiene retrocediendo a lo sumo 3 bytes.*

La autosincronización tiene una consecuencia práctica: si un archivo se corta por la mitad de un carácter, solo se pierde ese carácter, y los siguientes se decodifican bien [10, §2.3.2].

Además, UTF-8 es compatible con ASCII, porque los 128 primeros puntos de código ocupan un byte con su valor ASCII.

1.5 Consecuencias para el español y para los LLM

Observación 1.9. Las letras con tilde, la diéresis, la «ñ» y los signos de apertura «¿» y «¡» y las comillas angulares ocupan 2 bytes. En el fragmento inicial de *El Quijote* que se usa en la práctica del módulo hay 2484 caracteres y 2536 bytes; los 52 bytes de más corresponden a 48 letras con tilde o «ñ» y a 4 comillas angulares. La palabra «año» tiene 3 caracteres y 4 bytes.

Corolario 1.10. *Todo texto se representa sin pérdida como una cadena de $\mathcal{B}^*$, con $|\mathcal{B}| = 2^8 = 256$. Un tokenizador cuyo vocabulario inicial sean los 256 bytes nunca necesita un token «desconocido».*

El precio es que un carácter no ASCII ocupa varios tokens iniciales, y un tokenizador de bytes debe aprender fusiones como `C3` + `B1` → «ñ» antes de poder formar «año». Además, un token puede contener solo una parte de un carácter; el tema 2 analiza qué ocurre entonces al decodificar.

Tema 2

El algoritmo BPE

El algoritmo BPE (*Byte Pair Encoding*) construye un vocabulario de subpalabras de abajo arriba: parte de los símbolos básicos y fusiona, una y otra vez, el par adyacente más frecuente. Lo propuso Gage como método de compresión de datos [5], y Sennrich, Haddow y Birch lo adaptaron a la segmentación de palabras para la traducción automática [20, §3.2]; Radford *et al.* lo aplicaron a bytes en GPT-2 [19, §2.2]. Este tema define el algoritmo, demuestra cuánto acorta cada fusión y prueba que la decodificación recupera el texto. Se sigue [10, §2.4] y la §2.4 de la tarea 1 de CS336 [4].

2.1 Entrenamiento

Definición 2.1 (Corpus de palabras y pares). Un *corpus de entrenamiento* es un conjunto finito de palabras w, cada una con una frecuencia $n_w \geq 1$ y escrita como una secuencia de símbolos de un vocabulario $\mathcal{V}$. Para dos símbolos $a, b \in \mathcal{V}$, la *frecuencia del par* (a, b) es

$$N(a,b) = \sum_{w} n_w \cdot \#\{\, i : w_i = a,\ w_{i+1} = b \,\},$$

el número de posiciones en que b sigue inmediatamente a a, ponderado por la frecuencia de cada palabra.

Definición 2.2 (Fusión). La *fusión* del par (a, b) sustituye en cada palabra, recorriéndola de izquierda a derecha, cada aparición de a seguido de b por un símbolo nuevo $ab = a\|b$, y añade ab al vocabulario.

Definición 2.3 (Entrenamiento de BPE). Dados un corpus escrito en un vocabulario inicial $\mathcal{V}_0$ y un número de fusiones k, el *entrenamiento de BPE* repite k veces:

1. calcular $N(a, b)$ para todos los pares adyacentes;
2. elegir un par (a^*, b^*) de frecuencia máxima, con una regla de desempate fija;
3. fusionarlo y anotar la regla $a^* + b^* \to a^* b^*$.

El resultado es la *lista ordenada de fusiones* $\mu_1, \ldots, \mu_k$, que constituye el tokenizador.

Las palabras se obtienen del texto en un paso previo, la *pretokenización*, de modo que las fusiones nunca cruzan el límite entre dos palabras [20, §3.2]. Sennrich *et al.* añaden al final de cada palabra un símbolo especial de fin de palabra para poder reconstruir los espacios.

Proposición 2.4. *Tras k fusiones, el vocabulario tiene a lo sumo $|\mathcal{V}_0| + k$ símbolos, con igualdad cuando las k cadenas creadas son distintas entre sí y no pertenecen a $\mathcal{V}_0$. En particular, si $\mathcal{V}_0$ está formado por símbolos de longitud 1, $|\mathcal{V}| = |\mathcal{V}_0| + k$ salvo repeticiones de una misma cadena obtenida por dos caminos distintos.*

En la práctica se usa la igualdad $|\mathcal{V}| = |\mathcal{V}_0| + k$, y k es el único hiperparámetro del entrenamiento [20, §3.2].

Lema 2.5. *Si $a \neq b$, dos apariciones del par (a, b) en una secuencia no comparten ninguna posición.*

Proposición 2.6 (Compresión de una fusión). *Sea un corpus con L símbolos en total (contando cada palabra n_w veces) y sea (a, b), con $a \neq b$, un par de frecuencia $N(a, b) = c$. Tras fusionarlo, el corpus tiene $L - c$ símbolos.*

Observación 2.7. Si $a = b$, las apariciones pueden solaparse: «`aaa`» contiene dos pares $(\texttt{a}, \texttt{a})$, pero la fusión de izquierda a derecha solo puede aplicar una y deja «`aa·a`». En ese caso la longitud baja en el número de apariciones *no solapadas*. Por la proposición 2.6, elegir el par más frecuente es elegir la fusión que más acorta el corpus en ese paso: BPE es un algoritmo *voraz*. No garantiza la segmentación globalmente más corta con k fusiones, pero funciona bien en la práctica.

Cuadro 2.1: Cuatro fusiones de BPE sobre el corpus del ejemplo 2.8. El punto separa los símbolos de cada palabra.

Paso	Pares más frecuentes	Fusión	Corpus resultante	L
0	—	—	`c·a·s·a` (×3); `c·a·s·a·s` (×2); `m·a·s·a`; `m·e·s·a`	30
1	$(\texttt{a}, \texttt{s})$ 8, $(\texttt{s}, \texttt{a})$ 7	`a + s`	`c·as·a` (×3); `c·as·as` (×2); `m·as·a`; `m·e·s·a`	22
2	$(\texttt{c}, \texttt{as})$ 5, $(\texttt{as}, \texttt{a})$ 4	`c + as`	`cas·a` (×3); `cas·as` (×2); `m·as·a`; `m·e·s·a`	17
3	$(\texttt{cas}, \texttt{a})$ 3, $(\texttt{cas}, \texttt{as})$ 2	`cas + a`	`casa` (×3); `cas·as` (×2); `m·as·a`; `m·e·s·a`	14
4	$(\texttt{cas}, \texttt{as})$ 2, resto 1	`cas + as`	`casa` (×3); `casas` (×2); `m·as·a`; `m·e·s·a`	12

2.2 Codificación y decodificación

Definición 2.9 (Codificación BPE). Para codificar una palabra nueva se escribe en símbolos de $\mathcal{V}_0$ y se aplican las fusiones $\mu_1, \ldots, \mu_k$ *en el orden en que se aprendieron*, cada una a toda la palabra, de izquierda a derecha. La *decodificación* de una secuencia de tokens es la concatenación de las cadenas que representan.

Teorema 2.11 (Corrección de BPE). *Sea $\mathcal{V}_0$ un vocabulario inicial tal que toda cadena admite una escritura en símbolos de $\mathcal{V}_0$, y sea δ la decodificación por concatenación. Para toda cadena x, la codificación $\tau(x)$ está definida y $\delta(\tau(x)) = x$.*

Corolario 2.12. *Un tokenizador BPE nunca falla: en el peor caso no se aplica ninguna fusión y se devuelven los símbolos de $\mathcal{V}_0$. Con $\mathcal{V}_0 = \mathcal{B}$ (BPE a nivel de byte), por el corolario 1.10 no hay tokens desconocidos.*

2.3 Desempates y determinismo

Cuando varios pares empatan en frecuencia máxima, el paso 2 de la definición 2.3 necesita una regla. Cualquier regla sirve, pero debe ser fija para que el tokenizador sea reproducible. La tarea de CS336 elige el par mayor en orden lexicográfico [4, §2.4]; otra opción frecuente es el primer par que aparece en el corpus. Reglas distintas producen tokenizadores distintos (ejercicio 2.5).

Observación 2.13. La codificación de la definición 2.9 es determinista: la misma cadena da siempre la misma secuencia de tokens. El modelo unigram del tema 3, en cambio, define una distribución sobre segmentaciones y permite muestrearlas.

2.4 BPE a nivel de byte y pretokenización

GPT-2 aplica BPE a los bytes UTF-8 del texto, con $\mathcal{V}_0 = \mathcal{B}$ y $|\mathcal{V}_0| = 256$, en lugar de los más de $1{,}3 \cdot 10^5$ símbolos que exigiría partir de los puntos de código [19, §2.2]. Su vocabulario tiene 50 257 tokens [19, §2.3]: los 256 bytes, un token especial de fin de texto y 50 000 fusiones (ejercicio 2.2).

Definición 2.14 (Pretokenización). La *pretokenización* divide el texto en fragmentos mediante una expresión regular antes de aplicar BPE, y las fusiones se aplican dentro de cada fragmento [10, §2.6.9]. En GPT-2, cada fragmento es típicamente una palabra con su espacio delante (␣`lugar`, ␣`de`), un signo de puntuación o un grupo de dígitos; así se impide fusionar bytes de categorías distintas [19, §2.2].

Sin pretokenización, BPE aprende tokens que mezclan palabras y puntuación, como «`de␣su␣hacienda.␣`». Una consecuencia de la pretokenización es que ␣`hidalgo` y `hidalgo` son tokens distintos: un espacio de más al final de un texto cambia su tokenización.

Observación 2.15 (Tokens que contienen medio carácter). Con $\mathcal{V}_0 = \mathcal{B}$, un token puede ser un fragmento de un carácter multibyte, por ejemplo el byte `C3` solo. Por el teorema 2.11, la concatenación de todos los tokens de un texto es su cadena UTF-8, que se decodifica sin pérdida (teorema 1.7). Pero un token aislado puede no ser UTF-8 válido, y una interfaz que decodifica cada token al recibirlo debe guardar en un búfer los bytes de un carácter incompleto (ejercicio 2.7).

Tema 3

El modelo unigram, SentencePiece y el vocabulario

BPE construye el vocabulario de abajo arriba y codifica de forma determinista. El modelo unigram de Kudo [11] sigue el camino inverso: parte de un vocabulario grande, asigna una probabilidad a cada token, segmenta eligiendo la segmentación más probable y entrena podando. Este tema define el modelo, demuestra la corrección del algoritmo de Viterbi que lo segmenta, describe la biblioteca SentencePiece [12] y estudia el coste del vocabulario y los efectos de la tokenización sobre el comportamiento de los modelos. Se sigue [11, §3.2–3.4], [12, §3] y [10, §2.4.3].

3.1 El modelo unigram

Definición 3.1 (Modelo unigram de segmentación). Sea $\mathcal{V}$ un vocabulario de subpalabras y sea $p\colon \mathcal{V} \to (0,1]$ con $\sum_{t\in\mathcal{V}} p(t) = 1$. Una *segmentación* de una cadena x es una secuencia $s = (t_1, \dots, t_m)$ de tokens de $\mathcal{V}$ con $t_1 \| \dots \| t_m = x$; se denota $S(x)$ el conjunto de segmentaciones de x. El *modelo unigram* asigna a cada segmentación la probabilidad

$$P(s) = \prod_{i=1}^{m} p(t_i),$$

y la segmentación elegida es $s^* = \arg\max_{s\in S(x)} P(s)$ [11, §3.2].

El modelo es el de unigramas del módulo 1, ahora sobre subpalabras: los tokens se suponen independientes. Como cada factor es menor que 1, las segmentaciones con menos tokens tienden a ser más probables, igual que las palabras cortas en un modelo de bigramas.

Cuadro 3.1: Segmentaciones de «casas» con el vocabulario del ejemplo 3.2.

Segmentación	Producto	$P(s)$
`casa·s`	$0{,}2 \cdot 0{,}1$	0,02
`cas·as`	$0{,}1 \cdot 0{,}15$	0,015
`cas·a·s`	$0{,}1 \cdot 0{,}2 \cdot 0{,}1$	0,002
`c·as·as`	$0{,}05 \cdot 0{,}15 \cdot 0{,}15$	0,001125
`c·a·s·as` y `c·as·a·s`	$0{,}05 \cdot 0{,}2 \cdot 0{,}1 \cdot 0{,}15$	$1{,}5 \cdot 10^{-4}$ cada una
`c·a·s·a·s`	$0{,}05 \cdot 0{,}2^2 \cdot 0{,}1^2$	$2 \cdot 10^{-5}$

3.2 Segmentación óptima: el algoritmo de Viterbi

El número de segmentaciones crece exponencialmente con la longitud: una cadena de n caracteres tiene hasta 2^{n-1} (ejercicio 3.7). La enumeración del cuadro 3.1 no es viable para textos

reales, pero el problema tiene subestructura óptima.

Teorema 3.3 (Viterbi para el modelo unigram). *Sea $x = x_1 \dots x_n$ y sea $\alpha(j)$ la probabilidad máxima de una segmentación del prefijo $x_{1:j}$, con $\alpha(0) = 1$ y $\alpha(j) = 0$ si el prefijo no admite segmentación. Entonces, para $j = 1, \dots, n$,*

$$\alpha(j) = \max_{\substack{0 \le i < j \\ x_{i+1:j} \in \mathcal{V}}} \alpha(i)\, p(x_{i+1:j}),$$

y $\alpha(n) = \max_{s \in S(x)} P(s)$. Si ℓ es la longitud máxima de un token, el cálculo requiere $O(n\ell)$ productos.

3.3 Entrenamiento por EM y poda

Las probabilidades $p(t)$ no se observan, porque el corpus no viene segmentado. Kudo trata la segmentación como una variable oculta y maximiza la verosimilitud marginal [11, §3.2]

$$\mathcal{L} = \sum_{x \in \mathcal{D}} \ln \sum_{s \in S(x)} P(s),$$

donde $\mathcal{D}$ es el corpus de frases. El algoritmo EM alterna dos pasos: el paso E calcula, con las probabilidades actuales, el número esperado de veces que aparece cada token en las segmentaciones de cada frase, y el paso M reestima $p(t)$ como esos recuentos esperados normalizados. Es la misma idea que el estimador de máxima verosimilitud por frecuencias relativas del módulo 1, con recuentos esperados en lugar de observados.

El vocabulario se obtiene podando [11, §3.2]:

1. se parte de un vocabulario semilla grande: todos los caracteres y las subcadenas frecuentes del corpus;
2. con el vocabulario fijo, se optimizan las $p(t)$ por EM;
3. para cada token se estima cuánto bajaría $\mathcal{L}$ si se eliminara, y se conserva el $\eta\,\%$ de los tokens con mayor pérdida (por ejemplo, el 80 %);
4. se repiten los pasos 2 y 3 hasta alcanzar el tamaño deseado.

Los caracteres individuales nunca se eliminan, de modo que toda cadena sigue teniendo al menos una segmentación.

Observación 3.5 (Regularización por subpalabras). Como el modelo define una distribución sobre $S(x)$, durante el entrenamiento de un modelo neuronal se puede muestrear una segmentación distinta en cada paso, con probabilidad proporcional a $P(s)^{\alpha}$. Un α pequeño da muestras casi uniformes; uno grande se acerca a la segmentación de Viterbi. Kudo muestra que este ruido hace más robustos los modelos de traducción [11, §3.3].

3.4 SentencePiece

SentencePiece es una biblioteca que entrena tanto BPE como el modelo unigram directamente sobre frases sin pretokenizar [12]. Trata el texto como una secuencia de caracteres Unicode en la que el espacio es un símbolo más, que se escribe con el metasímbolo ▁ (`U+2581`); así, «Hello

Cuadro 3.2: Comparación de BPE y del modelo unigram [11, §3.4].

	BPE	Unigram
Construcción	de abajo arriba (fusiona)	de arriba abajo (poda)
Resultado del entrenamiento	lista ordenada de fusiones	probabilidad de cada token
Codificación	aplicar las fusiones en orden	Viterbi: segmentación más probable
Varias segmentaciones	no; es determinista	sí; se pueden muestrear

world.» puede segmentarse como `[Hello] [_wor] [ld] [.]` [12, §3.1]. Como el espacio forma parte de los tokens, decodificar consiste en concatenar y sustituir _ por un espacio, y la tokenización es *sin pérdida*:

$$\mathrm{dec}(\mathrm{enc}(\mathcal{N}(x))) = \mathcal{N}(x),$$

donde $\mathcal{N}$ es la normalización del texto, por defecto una variante de la forma NFKC de Unicode [12, §3.4]. Al no depender de los espacios para pretokenizar, el método se aplica igual a idiomas que no separan las palabras, como el japonés. En la literatura, «SentencePiece» se usa a veces como sinónimo de tokenización unigram [10, §2.4.3].

3.5 El tamaño del vocabulario

Proposición 3.6. *Un modelo con vocabulario de tamaño K y dimensión d tiene Kd parámetros en la matriz de embeddings de entrada y otros Kd en la matriz de salida que produce los logits, salvo que ambas se compartan. El coste es lineal en K.*

Cuadro 3.3: Coste de la matriz de embeddings en modelos con datos públicos.

Modelo	K	d	Kd
GPT-2 pequeño (2019) [19]	50 257	768	38 597 376
Llama 2 7B (2023) [21]	32 000	4096	131 072 000
Llama 3 8B (2024) [14]	128 256	4096	525 336 576

En Llama 3 8B, solo la matriz de entrada tiene unos $5{,}25 \cdot 10^8$ parámetros, que en `bf16` (2 bytes por parámetro) ocupan 1,05 GB. Según la información publicada, Grok-1 usa 131 072 tokens y Gemma, 256 000.

Definición 3.7 (Compresión). La *compresión* de un tokenizador sobre un texto es el cociente entre el número de bytes UTF-8 del texto y el número de tokens que produce, medido en bytes por token.

Por la proposición 2.6, sobre el propio corpus de entrenamiento cada fusión reduce el número de tokens, de modo que la compresión crece con k; el aumento es cada vez menor, porque los pares que se fusionan son cada vez menos frecuentes. El cuadro 3.4 muestra los resultados de la práctica del módulo, con un BPE de bytes entrenado sobre el fragmento de 2536 bytes de *El Quijote*. Estas cifras sobre el texto de entrenamiento son optimistas: en la misma práctica, entrenando con los dos primeros párrafos y 200 fusiones, la compresión es de 2,616 bytes por token en el texto de entrenamiento y de 1,953 en el tercer párrafo, que el tokenizador no vio.

Cuadro 3.4: Compresión de un BPE de bytes sobre su propio corpus de entrenamiento (práctica del módulo).

Fusiones	0	10	25	50	100	200	300
Tokens	2536	2047	1751	1483	1230	982	819
Bytes por token	1,000	1,239	1,448	1,710	2,062	2,582	3,096

3.6 Efectos de la tokenización

Observación 3.8 (Letras). El modelo recibe enteros, no letras. Para contar las «r» de «ferrocarril», segmentada por ejemplo como `ferro` · `carril`, tendría que conocer la ortografía interna de cada token, que no figura en la entrada y solo se aprende de forma indirecta.

Observación 3.9 (Números). Si los números se segmentan de forma irregular, el mismo dígito ocupa posiciones distintas dentro de los tokens según el número, lo que dificulta la aritmética [10, ejercicio 2.2]. Por eso algunos tokenizadores pretrocean los números en grupos de hasta tres dígitos antes de aplicar BPE.

Observación 3.10 (Idiomas). Un tokenizador entrenado con poco texto de un idioma lo segmenta en muchos tokens. Con el tokenizador de la práctica, entrenado en español, «El perro ladra en la plaza.» (27 bytes) da 15 tokens; «The dog barks in the square.» (28 bytes), 21, y una frase rusa de 22 bytes, 22 tokens: ninguna fusión encaja con los bytes del alfabeto cirílico. Petrov *et al.* midieron, en tokenizadores comerciales, diferencias de longitud de hasta 15 veces para un mismo texto traducido [18]. El mismo mensaje cuesta más y cabe menos en el contexto.

Tema 4

Embeddings y word2vec

Un token es un entero sin estructura: el índice de «gato» no está más cerca del de «perro» que del de «avión». Un *embedding* asigna a cada token un vector de $\mathbb{R}^d$, de modo que la geometría pueda reflejar el significado. Este tema define la matriz de embeddings, presenta la hipótesis distribucional y los vectores de coocurrencia, desarrolla el modelo skip-gram de word2vec con softmax y con muestreo negativo, y demuestra que este último factoriza de forma implícita una matriz de información mutua puntual. Se sigue [10, §5.2–5.5], la clase 2 de CS224N [3] y los artículos de Mikolov *et al.* [15, 16] y de Levy y Goldberg [13].

4.1 La matriz de embeddings

Definición 4.1 (Matriz de embeddings). Sea $\mathcal{V}$ un vocabulario de tamaño K con índices $0, \dots, K-1$. Una *matriz de embeddings* es una matriz $E \in \mathbb{R}^{K \times d}$; el *embedding* del token i es su fila $E_{i,:} \in \mathbb{R}^{1 \times d}$.

Proposición 4.2. *Sea $e_i \in \mathbb{R}^K$ el vector* one-hot *del token i, con un 1 en la posición i y ceros en el resto. Entonces $e_i^\top E = E_{i,:}$.*

La proposición 4.2 muestra que la capa de embeddings es una capa lineal sin sesgo aplicada a una entrada *one-hot*. En la práctica no se multiplica, porque el producto costaría Kd operaciones y la indexación $E[i]$ no cuesta ninguna; pero la equivalencia explica cómo se calcula el gradiente: solo las filas de los tokens presentes reciben gradiente distinto de cero (módulo 9).

Definición 4.3 (Similitud coseno). Para $a, b \in \mathbb{R}^d$ no nulos, la *similitud coseno* es $\cos(a, b) = \dfrac{a \cdot b}{\|a\| \, \|b\|} \in [-1, 1]$.

Por la desigualdad de Cauchy–Schwarz (módulo 2), $|\cos(a, b)| \le 1$, con igualdad si y solo si los vectores son proporcionales. El coseno no cambia al multiplicar un vector por una constante positiva (ejercicio 4.3), y por eso no penaliza que una palabra sea más frecuente que otra [10, §5.4].

4.2 La hipótesis distribucional

La *hipótesis distribucional* afirma que las palabras que aparecen en contextos parecidos tienen significados parecidos; Se atribuye a Firth la formulación según la cual una palabra se conoce por las palabras que la acompañan, que la tarea 2 de CS224N toma como punto de partida de word2vec [3]; véase también [10, §5.2]. Si «gato» y «perro» aparecen junto a «come», «duerme» y «pelo», sus representaciones deben parecerse.

Definición 4.4 (Matriz de coocurrencias). Fijada una ventana de m palabras, la *matriz de coocurrencias* $N \in \mathbb{R}^{K\times K}$ tiene por entrada $N(w,c)$ el número de veces que la palabra c aparece a distancia a lo sumo m de w en el corpus. La fila $N(w,:)$ es una representación de w por recuentos [10, §5.3].

Los recuentos están dominados por las palabras frecuentes («de», «que»), que aparecen junto a todas. La información mutua puntual del módulo 6 corrige ese efecto comparando la frecuencia observada de un par con la esperada si las palabras fueran independientes.

Definición 4.5 (Información mutua puntual). Sean $P(w,c) = N(w,c)/|\mathcal{D}|$, $P(w) = \sum_c P(w,c)$ y $P(c) = \sum_w P(w,c)$, con $|\mathcal{D}| = \sum_{w,c} N(w,c)$ el número de pares. La *información mutua puntual* y su versión positiva son

$$\mathrm{PMI}(w,c) = \ln \frac{P(w,c)}{P(w)\,P(c)}, \qquad \mathrm{PPMI}(w,c) = \max\{\mathrm{PMI}(w,c), 0\}.$$

La PMI vale 0 si w y c son independientes, es positiva si coinciden más de lo esperado y vale $-\infty$ si el par no se observa nunca; la PPMI evita ese valor. La información mutua del módulo 6 es la esperanza de la PMI bajo $P(w,c)$.

4.3 Skip-gram

Los vectores de recuentos tienen dimensión K y casi todas sus entradas son nulas. Word2vec aprende directamente vectores densos de dimensión $d \ll K$ con una tarea de predicción [15].

Definición 4.6 (Pares de skip-gram). Sea una frase $w_1, \ldots, w_n$ y una ventana $m \geq 1$. Los *pares de entrenamiento* de skip-gram son los pares ordenados (centro, contexto) (w_t, w_{t+j}) con $1 \leq |j| \leq m$ y $1 \leq t+j \leq n$.

Proposición 4.7. *Si $n \geq m+1$, una frase de n palabras da $m(2n-m-1)$ pares con ventana m.*

Por ejemplo, «el gato come pescado» ($n=4$) da $1\cdot(8-1-1) = 6$ pares con $m=1$, (el, gato), (gato, el), (gato, come), (come, gato), (come, pescado) y (pescado, come), y $2\cdot(8-2-1) = 10$ con $m=2$.

Definición 4.8 (Skip-gram con softmax). Cada palabra w tiene dos vectores: $v_w \in \mathbb{R}^d$ cuando es centro y $u_w \in \mathbb{R}^d$ cuando es contexto. El modelo es

$$P(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w\in\mathcal{V}} \exp(u_w^\top v_c)} = \mathrm{softmax}(U^\top v_c)_o,$$

donde $U \in \mathbb{R}^{d\times K}$ tiene por columnas los u_w. La pérdida de un par es $J = -\ln P(o \mid c)$, la entropía cruzada entre el vector *one-hot* y de o y la predicción $\hat{y} = \mathrm{softmax}(U^\top v_c)$ [3].

Proposición 4.9 (Gradiente respecto al vector central).

$$\frac{\partial J}{\partial v_c} = \sum_{w\in\mathcal{V}} P(w \mid c)\, u_w - u_o.$$

El gradiente es «lo esperado menos lo observado»: el descenso de gradiente acerca v_c al vector del contexto real y lo aleja de la media de los vectores de contexto que el modelo predice. Calcular esa media cuesta $O(Kd)$ por par, demasiado para $K \sim 10^5$ y miles de millones de pares.

4.4 Muestreo negativo

Definición 4.10 (Skip-gram con muestreo negativo). Sean w la palabra central, c_+ un contexto real y $c_1, \dots, c_k$ palabras de ruido extraídas al azar. La pérdida de *skip-gram con muestreo negativo* (SGNS) es

$$L = -\ln \sigma(c_+ \cdot w) - \sum_{i=1}^{k} \ln \sigma(-c_i \cdot w),$$

donde w denota el vector central y c_+, c_i los vectores de contexto [10, §5.5.2]. Mikolov *et al.* extraen el ruido de la distribución unigram elevada a $3/4$ y renormalizada [16].

La softmax sobre K palabras se sustituye por $k+1$ regresiones logísticas (módulo 8): distinguir el par real de k pares falsos. El coste por par pasa a $O(kd)$, con k entre 2 y 20.

Proposición 4.11 (Gradientes de SGNS).

$$\frac{\partial L}{\partial c_+} = \big(\sigma(c_+ \cdot w) - 1\big) w, \qquad \frac{\partial L}{\partial c_i} = \sigma(c_i \cdot w)\, w, \qquad \frac{\partial L}{\partial w} = \big(\sigma(c_+ \cdot w) - 1\big) c_+ + \sum_{i=1}^{k} \sigma(c_i \cdot w)\, c_i.$$

La demostración es el ejercicio 4.6. Los signos confirman la intuición: el paso de descenso aumenta $c_+ \cdot w$ y disminuye cada $c_i \cdot w$.

4.5 Muestreo negativo y PMI

Teorema 4.13 (Levy y Goldberg). *Sea $N(w,c)$ el número de veces que el par (w,c) aparece en el corpus, $N(w)$ y $N(c)$ sus marginales y $|\mathcal{D}|$ el número total de pares, y supóngase que los k negativos de cada par se extraen de la distribución unigram de contextos $P(c) = N(c)/|\mathcal{D}|$. Si la dimensión es suficiente para que cada producto $x = u_c \cdot v_w$ pueda elegirse libremente, la pérdida total esperada de SGNS se minimiza en*

$$u_c \cdot v_w = \mathrm{PMI}(w,c) - \ln k$$

para todo par con $N(w,c) > 0$.

Observación 4.14. El teorema 4.13 une los dos enfoques de este tema: el modelo predictivo de word2vec aprende, en el óptimo, la misma información que la matriz de PMI de los métodos de recuentos, comprimida en vectores de dimensión d. Es la información mutua del módulo 6 convertida en geometría.

4.6 Embeddings en un LLM

En un LLM, la matriz E se aprende junto con el resto del modelo, sin entrenamiento previo con word2vec, y a menudo se comparte con la matriz de salida (*weight tying*) [10, §14.2.3]. La intuición distribucional se mantiene: tras el entrenamiento, tokens con usos parecidos tienen embeddings cercanos. Pero un embedding de esta tabla es *estático*: «banco» tiene el mismo vector en «banco de peces» y en «Banco de España». En el módulo 13 la atención mezcla los vectores según el contexto y produce representaciones contextuales.

Tema 5

Modelos secuenciales: n-gramas, RNN y LSTM

Con tokens y embeddings ya definidos, queda modelar la dependencia entre posiciones. Este tema repasa los tres modelos que precedieron al Transformer y demuestra por qué ninguno escala: los n-gramas no generalizan, las redes recurrentes sufren el desvanecimiento del gradiente y todas las arquitecturas recurrentes son secuenciales en el tiempo. Se sigue [10, cap. 14], la clase 4 de CS224N [3], [8, cap. 10] y [23, §9.7 y §10.1]; los resultados sobre el gradiente proceden de Bengio, Simard y Frasconi [2] y de Pascanu, Mikolov y Bengio [17, §2–3], y la LSTM, de Hochreiter y Schmidhuber [9].

5.1 N-gramas y dispersión

Un modelo de n-gramas (módulo 1) estima $P(x_t \mid x_{t-n+1:t-1})$ por frecuencias relativas y necesita una tabla de K^n entradas.

Proposición 5.1. *En un corpus de $N \geq n$ tokens aparecen a lo sumo $N - n + 1$ n-gramas distintos. Por tanto, la fracción de casillas de la tabla con recuento positivo es a lo sumo $(N - n + 1)/K^n$.*

El defecto de fondo no es solo el tamaño: cada contexto es una casilla aislada y el modelo no sabe que «el gato come» y «el perro come» se parecen. El modelo neuronal de Bengio *et al.* [1], estudiado en el módulo 9, resuelve la generalización con embeddings, pero conserva un contexto de longitud fija.

5.2 Redes recurrentes

Definición 5.3 (Red recurrente). Sean $x_1, \dots, x_T \in \mathbb{R}^d$ los embeddings de una secuencia. Una *red recurrente* (RNN) simple con estado de dimensión h calcula

$$h_t = \tanh(W h_{t-1} + U x_t + b), \qquad z_t = W_{\mathrm{s}}\, h_t + b_{\mathrm{s}},$$

con $h_0 = 0$, $W \in \mathbb{R}^{h \times h}$, $U \in \mathbb{R}^{h \times d}$, $b \in \mathbb{R}^h$ y una capa de salida $W_{\mathrm{s}} \in \mathbb{R}^{K \times h}$ que produce los logits z_t del token siguiente [10, §14.1–14.2].

Los mismos parámetros se usan en todas las posiciones. El estado h_t resume todo el prefijo $x_{1:t}$, de modo que el contexto no tiene, en principio, longitud fija.

Proposición 5.4. *La parte recurrente de una RNN simple tiene $h(h + d) + h$ parámetros, independientemente de T.*

5.3 Retropropagación en el tiempo y desvanecimiento del gradiente

El gradiente de una RNN se calcula aplicando la retropropagación al grafo desplegado de la figura 5.1; el procedimiento se llama *retropropagación en el tiempo* [23, §9.7].

Lema 5.5. *Sea $a_t = W h_{t-1} + U x_t + b$. El jacobiano de un paso es $\dfrac{\partial h_t}{\partial h_{t-1}} = D_t W$, con $D_t = \operatorname{diag}\big(1 - \tanh^2(a_t)\big)$, y*

$$\frac{\partial h_T}{\partial h_0} = D_T W \, D_{T-1} W \cdots D_1 W.$$

Teorema 5.6 (Cota exponencial del gradiente). *Sea $\|\cdot\|$ la norma espectral. En la RNN de la definición 5.3,*

$$\left\| \frac{\partial h_T}{\partial h_0} \right\| \le \|W\|^T.$$

En particular, si $\|W\| \le \gamma < 1$, el gradiente que llega a h_0 desde la pérdida en la posición T decae al menos como γ^T.

Corolario 5.7. *Para que el gradiente pueda crecer exponencialmente (*explotar*) es necesario que $\|W\| > 1$. Con una activación de derivada acotada por γ_ϕ (por ejemplo, $\gamma_\phi = 1/4$ para la sigmoide), la condición necesaria es $\|W\| > 1/\gamma_\phi$ [17, §2.1].*

Cuadro 5.1: Factor w^T en la RNN escalar lineal $h_t = w\, h_{t-1}$.

w	$T = 10$	$T = 50$	$T = 100$
0,9	0,3487	$5{,}154 \cdot 10^{-3}$	$2{,}656 \cdot 10^{-5}$
0,99	0,9044	0,6050	0,3660
1,1	2,594	117,4	13 781

El caso escalar lineal del cuadro 5.1 muestra el fenómeno: con $w = 0{,}9$, la señal de un token situado 100 posiciones atrás llega multiplicada por $2{,}7 \cdot 10^{-5}$, y la red no aprende dependencias largas. Bengio, Simard y Frasconi demostraron que, para que una red recurrente almacene información de forma robusta frente al ruido, debe operar en una región en la que el gradiente se desvanece, de modo que aprender dependencias largas por descenso de gradiente es intrínsecamente difícil [2].

Definición 5.8 (Recorte del gradiente). Dado un umbral $\tau > 0$, el *recorte por norma* sustituye el gradiente g por

$$\operatorname{clip}_\tau(g) = \begin{cases} g & \text{si } \|g\| < \tau, \\ \tau\, g / \|g\| & \text{si } \|g\| \ge \tau. \end{cases}$$

Pascanu *et al.* proponen esta regla contra la explosión del gradiente y sugieren un umbral entre la mitad y diez veces la norma media del gradiente [17, §3.2].

Proposición 5.9. *$\|\operatorname{clip}_\tau(g)\| \le \tau$, y $\operatorname{clip}_\tau(g)$ es un múltiplo positivo de g; en particular, $g \cdot \operatorname{clip}_\tau(g) \ge 0$ y el paso recortado sigue siendo de descenso.*

El recorte resuelve la explosión, pero no el desvanecimiento: un gradiente pequeño no se agranda.

5.4 La LSTM

Definición 5.10 (LSTM). Una LSTM añade al estado oculto h_t una *celda de memoria* $c_t \in \mathbb{R}^h$ y tres *puertas* con valores en $(0,1)^h$:

$$\begin{aligned} f_t &= \sigma(W_f h_{t-1} + U_f x_t + b_f), & i_t &= \sigma(W_i h_{t-1} + U_i x_t + b_i), \\ o_t &= \sigma(W_o h_{t-1} + U_o x_t + b_o), & \tilde{c}_t &= \tanh(W_c h_{t-1} + U_c x_t + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

La puerta de *olvido* f_t decide qué parte de la memoria se conserva; la de *entrada* i_t, qué parte del candidato $\tilde{c}_t$ se añade, y la de *salida* o_t, qué parte de la memoria pasa al estado oculto [9], [10, §14.5], [23, §10.1].

La puerta de olvido no figuraba en el artículo original de 1997; la introdujeron Gers, Schmidhuber y Cummins [7], y es la forma que usan los libros de texto actuales.

Proposición 5.11. *El jacobiano de c_t respecto a c_{t-1} a través del camino directo, es decir, manteniendo fijas las puertas y el candidato, es* $\operatorname{diag}(f_t)$*. A lo largo de T pasos, el producto de estos jacobianos es* $\operatorname{diag}\big(\prod_{t=1}^{T} f_t\big)$*, con el producto componente a componente.*

La celda se actualiza *sumando* y no multiplicando por una matriz W: si la red aprende $f_t \approx 1$, el gradiente recorre muchos pasos casi sin atenuarse. Con $f_t = 0{,}98$ constante, el factor a 100 pasos es $0{,}98^{100} = 0{,}1326$, frente a $2{,}7 \cdot 10^{-5}$ en la RNN con $w = 0{,}9$. Es la misma idea que la conexión residual del módulo 11. En la práctica, las LSTM aprovechan contextos de cientos de tokens.

Corolario 5.12. *Una LSTM con estado de dimensión h y entrada de dimensión d tiene $4\big(h(h+d)+h\big)$ parámetros recurrentes, cuatro veces los de una RNN simple.*

5.5 Secuencialidad y el paso al Transformer

El segundo límite no tiene que ver con el gradiente. En una red recurrente, h_t depende de h_{t-1}, de modo que calcular h_T exige T pasos que no pueden ejecutarse a la vez dentro de una secuencia (figura 5.1). Una GPU tiene miles de núcleos que quedan ociosos durante esa cadena. El Transformer del módulo 13 calcula las representaciones de todas las posiciones de una capa a la vez, con productos de matrices, y conecta cualquier par de posiciones en un solo paso de atención, a cambio de un coste $O(n^2)$ en la longitud.

Cuadro 5.2: Modelos de secuencias y sus limitaciones.

Modelo	Contexto	Paralelo	Problema principal
N-grama	$n-1$ fijo	sí (contar)	tabla de K^n casillas; no generaliza
MLP (módulo 9)	fijo	sí	contexto corto y fijo
RNN	ilimitado en teoría	no	desvanecimiento del gradiente; secuencial
LSTM	cientos de tokens	no	secuencial; aprovecha mal la GPU
Transformer	ventana de contexto	sí	atención $O(n^2)$ (módulo 13)

Bibliografía

[1] Y. Bengio, R. Ducharme, P. Vincent y C. Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003. https://www.jmlr.org/papers/v3/bengio03a.html

[2] Y. Bengio, P. Simard y P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[3] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 2, «Word Vectors»; clase 4, «Language Models and RNNs»; tarea 2. https://web.stanford.edu/class/cs224n/

[4] Stanford CS336. Language Modeling from Scratch. Stanford University, primavera de 2026. Tarea 1 (*basics*): tokenizador BPE. https://github.com/stanford-cs336/assignment1-basics

[5] P. Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

[6] Gemini Team, Google. Gemini: A family of highly capable multimodal models. Informe técnico, 2023. https://arxiv.org/abs/2312.11805

[7] F. A. Gers, J. Schmidhuber y F. Cummins. Learning to forget: continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.

[8] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 10, «Sequence Modeling: Recurrent and Recursive Nets». https://www.deeplearningbook.org/

[9] S. Hochreiter y J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[10] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 2, «Words and Tokens»; cap. 5, «Embeddings»; cap. 14, «RNNs and LSTMs». https://web.stanford.edu/~jurafsky/slp3/

[11] T. Kudo. Subword regularization: improving neural network translation models with multiple subword candidates. En *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, págs. 66–75, 2018. https://aclanthology.org/P18-1007/

[12] T. Kudo y J. Richardson. SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing. En *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, págs. 66–71, 2018. https://aclanthology.org/D18-2012/

[13] O. Levy e Y. Goldberg. Neural word embedding as implicit matrix factorization. En *Advances in Neural Information Processing Systems 27 (NIPS)*, 2014.

[14] Llama Team, AI @ Meta. The Llama 3 herd of models. Informe técnico, 2024. https://arxiv.org/abs/2407.21783

[15] T. Mikolov, K. Chen, G. Corrado y J. Dean. Efficient estimation of word representations in vector space. *arXiv:1301.3781*, 2013. https://arxiv.org/abs/1301.3781

[16] T. Mikolov, I. Sutskever, K. Chen, G. Corrado y J. Dean. Distributed representations of words and phrases and their compositionality. En *Advances in Neural Information Processing Systems 26 (NIPS)*, 2013. https://arxiv.org/abs/1310.4546

[17] R. Pascanu, T. Mikolov e Y. Bengio. On the difficulty of training recurrent neural networks. En *Proceedings of the 30th International Conference on Machine Learning (ICML)*, PMLR 28, 2013. https://proceedings.mlr.press/v28/pascanu13.html

[18] A. Petrov, E. La Malfa, P. H. S. Torr y A. Bibi. Language model tokenizers introduce unfairness between languages. En *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. https://arxiv.org/abs/2305.15425

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[20] R. Sennrich, B. Haddow y A. Birch. Neural machine translation of rare words with subword units. En *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, págs. 1715–1725, 2016. https://aclanthology.org/P16-1162/

[21] H. Touvron *et al.* Llama 2: open foundation and fine-tuned chat models. Informe técnico, 2023. https://arxiv.org/abs/2307.09288

[22] F. Yergeau. UTF-8, a transformation format of ISO 10646. RFC 3629, Internet Engineering Task Force, noviembre de 2003. https://www.rfc-editor.org/rfc/rfc3629

[23] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §9.7, §10.1 y §15.6. https://d2l.ai/